## Testing Task 3

```python
random.seed(2)

sword = Weapon("Sword", Dice(20))
print(sword)
print("Five sword rolls:", [sword.roll_damage() for _ in range(5)])

armed = Character("Armed knight", weapon=sword)
unarmed = Character("Unarmed peasant")
dummy = Character("Training dummy", hp_max=1000)

print("Armed damage per attack:  ", [armed.attack(dummy) for _ in range(5)])
print("Unarmed damage per attack:", [unarmed.attack(dummy) for _ in range(5)])
print("Dummy HP after 10 attacks:", dummy.hp)
```

```
Weapon(name='Sword', damage_dice=Dice(sides=20))
Five sword rolls: [2, 3, 3, 12, 6]
Armed damage per attack:   [10, 9, 20, 7, 20]
Unarmed damage per attack: [10, 10, 10, 10, 10]
Dummy HP after 10 attacks: 884
```

**Discussion of Task 3.** The armed character's damage varies with the dice roll (1 to 20 for a d20), while the unarmed character always deals the fixed default damage of 10. Both go through `take_damage()` on the target.

# Task 4: Character specialisation

Two subclasses extend `Character`:

`Warrior` (defensive specialist)

- New attribute `armor` (default 5).
- Overrides `take_damage()` so incoming damage is reduced by `armor` (never below 0) before it is applied.

`Mage` (resource-based attacker)

- New attributes `mana` and `mana_max` (both default 30). `mana` is a property that is kept between 0 and `mana_max`.
- Overrides `attack()`:
  - With **at least 10 mana**: spend 10 mana and cast a spell dealing **double** the base damage. The spell ignores any equipped weapon.
  - With **less than 10 mana**: fall back to the base attack (using the equipped weapon, if any) and restore 5 mana.

```python
show_class(Warrior)
print()
show_class(Mage)
print()
show_attributes(Warrior("Example warrior"))
print()
show_attributes(Mage("Example mage"))
```

```
Warrior (inherits from Character)
  - __init__  [method]
  - take_damage  [method]

Mage (inherits from Character)
  - SPELL_COST  [class attribute]
  - MANA_REGEN  [class attribute]
  - __init__  [method]
  - mana  [property]
  - attack  [method]

Attributes of Warrior name=Example warrior hp=100 damage=10 weapon=none:
  - name = 'Example warrior'
  - _hp = 100
  - hp_max = 100
  - damage = 10
  - weapon = None
  - armor = 5

Attributes of Mage name=Example mage hp=100 damage=10 weapon=none:
  - name = 'Example mage'
  - _hp = 100
  - hp_max = 100
  - damage = 10
  - weapon = None
  - mana_max = 30
  - _mana = 30
```

## Testing the Warrior

```python
warrior = Warrior("Conan", armor=5)
print("Start HP:", warrior.hp)

warrior.take_damage(3)   # fully absorbed by armor
print("After 3 damage:", warrior.hp)

warrior.take_damage(12)  # 12 - 5 = 7 gets through
print("After 12 damage:", warrior.hp)
```

Python

```
Start HP: 100
After 3 damage: 100
After 12 damage: 93
```

## Testing the Mage

```python
mage = Mage("Merlin", weapon=Weapon("Staff", Dice(10)))
dummy = Character("Training dummy", hp_max=1000)

for turn in range(1, 7):
    mana_before = mage.mana
    dmg = mage.attack(dummy)
    kind = "spell " if mana_before >= 10 else "staff "
    print(f"Turn {turn}: {kind}| damage {dmg:2d} | mana {mana_before:2d} -> {mage.mana:2d}")
```

Python

```
Turn 1: spell | damage 20 | mana 30 -> 20
Turn 2: spell | damage 20 | mana 20 -> 10
Turn 3: spell | damage 20 | mana 10 ->  0
Turn 4: staff | damage  1 | mana  0 ->  5
Turn 5: staff | damage 10 | mana  5 -> 10
Turn 6: spell | damage 20 | mana 10 ->  0
```

**Discussion of Task 4.** The warrior absorbed a hit of 3 completely and lost only 7 HP from a hit of 12. The mage cast three spells (20 damage each, 10 mana each), then fell back to the staff, whose damage varies with the dice roll and gave 5 mana back per attack. Once the mana reached 10 again, the next attack was a spell.

# Task 5: Running battles

`Battle` manages turn-based combat between a `hero` and an `enemy`.

- `play_round()`: the hero attacks the enemy, and if the enemy is still alive, the enemy attacks the hero.
- `simulate_battle()`: runs rounds in a `while` loop until one character is no longer alive, and returns the winner.

```python
show_class(Battle)
```

Python

```
Battle
  - __init__  [method]
  - play_round  [method]
  - simulate_battle  [method]
```

## Battle 1: Warrior with sword vs Mage with staff

```python
random.seed(42)
battle = Battle(Warrior("Conan", weapon=Weapon("Sword", Dice(20))),
                Mage("Merlin", weapon=Weapon("Staff", Dice(10))))
winner = battle.simulate_battle()
```

Python

```
Round 1: Conan 85 HP | Merlin 96 HP
Round 2: Conan 70 HP | Merlin 95 HP
Round 3: Conan 55 HP | Merlin 86 HP
Round 4: Conan 55 HP | Merlin 78 HP
Round 5: Conan 55 HP | Merlin 73 HP
Round 6: Conan 40 HP | Merlin 55 HP
Round 7: Conan 35 HP | Merlin 52 HP
Round 8: Conan 35 HP | Merlin 38 HP
Round 9: Conan 20 HP | Merlin 37 HP
Round 10: Conan 20 HP | Merlin 34 HP
Round 11: Conan 16 HP | Merlin 26 HP
Round 12: Conan 1 HP | Merlin 6 HP
Round 13: Conan 0 HP | Merlin 5 HP
Merlin wins after 13 rounds!
```

## Battle 2: Both unarmed (Warrior vs Mage)

```python
random.seed(42)
winner = Battle(Warrior("Conan"), Mage("Merlin")).simulate_battle()
```

Python

```
Round 1: Conan 85 HP | Merlin 90 HP
Round 2: Conan 70 HP | Merlin 80 HP
Round 3: Conan 55 HP | Merlin 70 HP
Round 4: Conan 50 HP | Merlin 60 HP
Round 5: Conan 45 HP | Merlin 50 HP
Round 6: Conan 30 HP | Merlin 40 HP
Round 7: Conan 25 HP | Merlin 30 HP
Round 8: Conan 20 HP | Merlin 20 HP
Round 9: Conan 5 HP | Merlin 10 HP
Round 10: Conan 5 HP | Merlin 0 HP
Conan wins after 10 rounds!
```

## Battle 3: Armed vs unarmed (plain characters)

```python
random.seed(42)
winner = Battle(Character("Armed Anna", weapon=Weapon("Axe", Dice(20))),
                Character("Unarmed Ulf")).simulate_battle()
```

Python

```
Round 1: Armed Anna 90 HP | Unarmed Ulf 96 HP
Round 2: Armed Anna 80 HP | Unarmed Ulf 95 HP
Round 3: Armed Anna 70 HP | Unarmed Ulf 86 HP
Round 4: Armed Anna 60 HP | Unarmed Ulf 78 HP
Round 5: Armed Anna 50 HP | Unarmed Ulf 70 HP
Round 6: Armed Anna 40 HP | Unarmed Ulf 65 HP
Round 7: Armed Anna 30 HP | Unarmed Ulf 61 HP
Round 8: Armed Anna 20 HP | Unarmed Ulf 43 HP
Round 9: Armed Anna 10 HP | Unarmed Ulf 40 HP
Round 10: Armed Anna 0 HP | Unarmed Ulf 21 HP
Unarmed Ulf wins after 10 rounds!
```

## Battle 4: Mage vs Mage

```python
random.seed(42)
winner = Battle(Mage("Merlin", weapon=Weapon("Staff", Dice(10))),
                Mage("Gandalf", weapon=Weapon("Staff", Dice(10)))).simulate_battle()
```

Python

```
Round 1: Merlin 80 HP | Gandalf 80 HP
Round 2: Merlin 60 HP | Gandalf 60 HP
Round 3: Merlin 40 HP | Gandalf 40 HP
Round 4: Merlin 39 HP | Gandalf 38 HP
Round 5: Merlin 35 HP | Gandalf 33 HP
Round 6: Merlin 15 HP | Gandalf 13 HP
Round 7: Merlin 12 HP | Gandalf 9 HP
Round 8: Merlin 3 HP | Gandalf 7 HP
Round 9: Merlin 3 HP | Gandalf 0 HP
Merlin wins after 9 rounds!
```

## Who wins more often?

A single battle is random, so we simulate many battles (without printing every round) and count the wins. The hero attacks first in each round, which gives the hero a small advantage, so we run both seatings.

```python
def win_rate(make_hero, make_enemy, n=2000):
    wins = 0
    for _ in range(n):
        hero, enemy = make_hero(), make_enemy()
        if Battle(hero, enemy).simulate_battle(verbose=False) is hero:
            wins += 1
    return wins / n


random.seed(7)
def make_warrior(): return Warrior("Conan", weapon=Weapon("Sword", Dice(20)))
def make_mage():    return Mage("Merlin", weapon=Weapon("Staff", Dice(10)))

print(f"Warrior wins when attacking first: {win_rate(make_warrior, make_mage):.1%}")
print(f"Mage wins when attacking first:    {win_rate(make_mage, make_warrior):.1%}")
```

Python

```
Warrior wins when attacking first: 94.8%
Mage wins when attacking first:    11.5%
```

## Task 3: Items – the Weapon class

`Weapon` lives in `items.py` (not `weapons.py`) so the module can later hold other items too. A weapon has a `name` and a `damage_dice` (a `Dice` object), and `roll_damage()` rolls that dice.

This is **object composition**: a `Weapon` *has a* `Dice`, and a `Character` *has a* `Weapon`.

`Character.__init__` now takes an optional `weapon`. In `attack()`, an armed character deals damage rolled from the weapon; an unarmed character deals its default `damage`.

```python
show_class(Weapon)
print()
sword = Weapon("Sword", Dice(20))
show_attributes(sword)
```

Python

```
Weapon
  - __init__  [method]
  - roll_damage  [method]
  - __repr__  [method]

Attributes of Weapon(name='Sword', damage_dice=Dice(sides=20)):
  - name = 'Sword'
  - damage_dice = Dice(sides=20)
```

```python
import random
import inspect

from dice import Dice
from items import Weapon
from character import Character, Warrior, Mage
from battle import Battle


def show_class(cls):
    """Print the public methods and properties a class provides."""
    print(f"{cls.__name__}" + (f" (inherits from {cls.__bases__[0].__name__})" if cls.__bases__[0] is not object else ""))
    for name, member in vars(cls).items():
        if name.startswith("__") and name not in ("__init__", "__repr__", "__call__"):
            continue
        kind = "property" if isinstance(member, property) else ("class attribute" if isinstance(member, (int, float, str)) else "method")
        print(f"  - {name}  [{kind}]")


def show_attributes(obj):
    """Print the attributes stored on an object instance."""
    print(f"Attributes of {obj!r}:")
    for name, value in vars(obj).items():
        print(f"  - {name} = {value!r}")
```

Python